

```
// خروجی:
// پراید: عوارض 5000 تومان , حداکثر سرعت 180
// کامیون ولوو: عوارض 15000 تومان , حداکثر سرعت 100
// هوندا: عوارض 2000 تومان , حداکثر سرعت 220
```

Replace Type Code with Subclasses تموم شد! 🎉

حالا بریم سراغ آخرین درمان Primitive Obsession:
 Replace Type Code with State/Strategy

<https://refactoring.guru/replace-type-code-with-state-strategy>

.Link reading is unavailable in Expert Mode. Please use Instant Mode

Replace Type Code with State/Strategy

تعریف ساده

مشابه **Replace Type Code with Subclasses**، ولی وقتی استفاده میشه که نوع (Type Code) در طول زمان تغییر می‌کنه یا کلاس والد از قبل به دلیل دیگه‌ای نمیتونه زیرکلاس بشه.

VS تفاوت با Subclasses

State/Strategy		Subclasses	
می‌تونه تغییر کنه 		ثابته 	
Has-A (ترکیب)		Is-A (ارث‌بری)	
نوع می‌تونه بارها عوض بشه		نوع در زمان ساخت مشخص و ثابته	
		رابطه	
		کاربرد	

مثال: کی نیاز به State داریم؟

فرض کن یه سفارش داری که وضعیتش مدام عوض میشه:

csharp

Pending → Shipped → Delivered

با Subclasses همیشه این کار رو کرد! چون نمیتونی یه شیء ShippedOrder رو تبدیل کنی به DeliveredOrder. باید **State Pattern** رو استفاده کنی.

مثال قبل از ریفتور

csharp

```
public class Order
{
    ;public const int PENDING = 0
    ;public const int SHIPPED = 1
    ;public const int DELIVERED = 2
    ;public const int CANCELLED = 3

    ;public int Status { get; set; } = PENDING

    ()public string GetStatusName
    {
        switch (Status)
        {
            case PENDING: return "در انتظار";
            case SHIPPED: return "ارسال شده";
            case DELIVERED: return "تحویل شده";
            case CANCELLED: return "لغو شده";
            default: throw new Exception("نامعتبر");
        }
    }

    ()public void NextStatus
    {
        switch (Status)
        {
            case PENDING: Status = SHIPPED; break
```

```

        ;case SHIPPED: Status = DELIVERED; break
;("تحويل شده تغيير نمي‌کنه")case DELIVERED: throw new Exception
;("لغو شده تغيير نمي‌کنه")case CANCELLED: throw new Exception
    {
        {

        ()public bool CanCancel
        }

        switch (Status)
        {
            ;case PENDING: return true
            ;case SHIPPED: return false
            ;case DELIVERED: return false
            ;case CANCELLED: return false
            ;("نامعتبر")default: throw new Exception
        }
        {
            {
        }
    }

```

Replace Type Code with State/Strategy بعد از

csharp

```

        State Interface //
        public interface IOrderState
        {
            ;()string GetName
            ;()IOrderState Next
            ;()bool CanCancel
        }

        State های مختلف //
        public class PendingState : IOrderState
        {
            ;()string GetName => "در انتظار"
            ;()public IOrderState Next() => new ShippedState
            ;public bool CanCancel() => true
        }

        public class ShippedState : IOrderState
        {

```

```

        ;ارسال شده" <= ()public string GetName
;()public IOrderState Next() => new DeliveredState
        ;public bool CanCancel() => false
    }

    public class DeliveredState : IOrderState
    {
        ;تحويل شده" <= ()public string GetName
;("تحويل شده تغيير نمي‌کند")public IOrderState Next() => throw new Exception
        ;public bool CanCancel() => false
    }

    public class CancelledState : IOrderState
    {
        ;لغو شده" <= ()public string GetName
;("لغو شده تغيير نمي‌کند")public IOrderState Next() => throw new Exception
        ;public bool CanCancel() => false
    }

    // کلاس اصلی (حالا تمیزه!)
    public class Order
    {
        ;()private IOrderState _state = new PendingState

        ;()public string StatusName => _state.GetName
        ;()public bool CanCancel => _state.CanCancel

        ()public void NextStatus
        {
            ;()state = _state.Next_
        }
    }

```



مزایا

توضیح

مزیت

state_ میتونه عوض بشه

نوع قابل تغییر

هر State منطق خودش رو داره

بدون switch


```

        switch (State)
        {
            ;case DRAFT: State = REVIEW; break
            ;case REVIEW: State = PUBLISHED; break
            ;("منتشر شده تغییر نمی‌کند")case PUBLISHED: throw new Exception
            {
                {

                ()public bool CanEdit
                }
                switch (State)
                {
                    ;case DRAFT: return true
                    ;case REVIEW: return true
                    ;case PUBLISHED: return false
                    ;("نامعتبر")default: throw new Exception
                    {
                        {
                            {

```

راهنمایی:

- IDocumentState با متدهای GetName , Next , CanEdit
- DraftState , ReviewState , PublishedState

بفرست تا بررسی کنیم! 📩